ML revision sheet (before end sem)

Bias and variance

error = bias^2 + variance + irreducible noise.
high bias = underfitting (model too simple). high variance = overfitting (model too flexible, memorises the noise).
more complex model -> bias down, variance up. we want the sweet spot in the middle.

Regularization

L2 (ridge): adds lambda * sum of w^2 to the loss. shrinks weights towards 0 but rarely makes them exactly 0.
L1 (lasso): adds lambda * sum of |w|. can make weights exactly 0 so it does feature selection.
lambda very big -> underfit. lambda = 0 -> no regularization at all.
dropout: randomly drop units while training (eg p = 0.5). early stopping: stop when val loss starts going up.

Cross validation

k-fold: split the data into k parts, train on k-1, test on 1, repeat k times and average. usually k = 5 or 10.
LOOCV = k = n, so very expensive.
dont tune hyperparams on the test set!! the test set is only used once at the very end.

Classification metrics

precision = TP / (TP + FP). recall = TP / (TP + FN). F1 = harmonic mean of the two = 2PR / (P + R).
accuracy is misleading when classes are imbalanced (eg 99% negatives).
eg TP = 40, FP = 10, FN = 20 -> P = 0.8, R = 0.667, F1 = 0.727.
ROC curve = TPR vs FPR at different thresholds. AUC = area under it. 0.5 = random guessing.

Gradient descent bits

learning rate too big -> overshoots or diverges. too small -> very slow.
SGD uses 1 sample per update, mini-batch uses 32 to 256 samples (typical), batch uses all of them.
momentum keeps a running average of past gradients (beta = 0.9 usually).
Adam = momentum + RMSprop style scaling. defaults: beta1 = 0.9, beta2 = 0.999, eps = 1e-8.

Decision trees

split on the feature that gives the biggest drop in impurity. entropy = - sum of p log2 p. gini = 1 - sum of p^2.
information gain = entropy before the split minus the weighted entropy after it.
eg a node with 9 yes and 5 no has entropy 0.940 bits. a pure node has entropy 0.
trees overfit easily: fix with pruning, max depth, or a minimum number of samples per leaf. random forest = many trees on bootstrap samples + random feature subsets, then vote.

kNN and naive bayes

kNN: lazy learner, no training, stores the data. predicts by majority vote of the k nearest points. k small -> high variance, k large -> high bias.
needs feature scaling because distance is dominated by big-valued features. slow at prediction time and bad in high dimensions (curse of dimensionality).
naive bayes: assumes features are independent given the class. P(class | x) is proportional to P(class) * product of P(x_i | class).
laplace smoothing adds 1 to every count so an unseen word does not give probability 0.

Clustering and PCA

k-means: pick k centres, assign every point to the nearest centre, move each centre to the mean of its points, repeat until nothing changes.
converges but only to a local minimum so run it several times w/ different starts. k chosen with the elbow method (plot within-cluster sum of squares vs k).
PCA: finds the directions of maximum variance (eigenvectors of the covariance matrix), keeps the top few. use to reduce dimensions or to visualise.
always standardise the features before PCA. the first principal component is the direction with the most variance.

Neural net notes

perceptron can only learn linearly separable problems (cant do XOR). one hidden layer with a nonlinearity fixes that.
activation functions: sigmoid squashes to (0, 1) but saturates, tanh to (-1, 1), ReLU = max(0, x) is the default for hidden layers.
backprop = chain rule applied layer by layer from the loss back to the weights.
vanishing gradient: with sigmoid in deep nets the gradients shrink toward 0 in early layers. ReLU and residual connections help.
batch norm normalises each layers inputs over the mini-batch, lets us use bigger learning rates.